

11

GRADIENT DESCENT



In this final chapter, we'll slow down a bit and consider gradient descent afresh. We'll begin by reviewing the idea of gradient descent using illustrations, discussing what it is and how it works. Next, we'll explore the meaning of *stochastic* in *stochastic gradient descent*. Gradient descent is a simple algorithm that invites tweaking, so after we explore stochastic gradient descent, we'll consider a useful and commonly used tweak: momentum. We'll conclude the chapter by discussing more advanced, adaptive gradient descent algorithms, specifically RMSprop, Adagrad, and Adam.

This is a math book, but gradient descent is very much applied math, so we'll learn by experimentation. The equations are straightforward, and the math we saw in previous chapters is relevant as background. Therefore, consider this chapter an opportunity to apply what we've learned so far.

The Basic Idea

We've encountered gradient descent several times already. We know the form of the basic gradient descent update equations from Equation 10.14:

$$\begin{aligned}W &\leftarrow W - \eta \Delta W \\b &\leftarrow b - \eta \Delta b\end{aligned}\tag{11.1}$$

Here, ΔW and Δb are errors based on the partial derivatives of the weights and biases, respectively; η (eta) is a step size or learning rate, a value we use to adjust how we move.

Equation 11.1 isn't specific to machine learning. We can use the same form to implement gradient descent on arbitrary functions. Let's discuss gradient descent using 1D and 2D examples to lay a foundation for how it operates. We'll use an unmodified form of gradient descent known as *vanilla gradient descent*.

Gradient Descent in One Dimension

Let's begin with a scalar function of x :

$$f(x) = 6x^2 - 12x + 3\tag{11.2}$$

Equation 11.2 is a parabola facing upward. Therefore, it has a minimum. Let's find the minimum analytically by setting the derivative to zero and solving for x :

$$\frac{d}{dx} (6x^2 - 12x + 3) = 12x - 12 = 0$$

$$12x = 12$$

$$x = 1$$

The minimum of the parabola is at $x = 1$. Now, let's instead use gradient descent to find the minimum of Equation 11.2. How should we begin?

First, we need to write the proper update equation, the form of Equation 11.1 that applies in this case. We need the gradient, which for a 1D function is simply the derivative, $f'(x) = 12x - 12$. With the derivative, gradient descent becomes

$$x \leftarrow x - \eta f'(x) = x - \eta (12x - 12)\tag{11.3}$$

Notice that we subtract $\eta (12x - 12)$. This is why the algorithm is called gradient *descent*. Recall that the gradient points in the direction of maximum change in the function's value. We're interested in minimizing the function, not maximizing it, so we move in the direction opposite to the gradient toward smaller function values; therefore, we subtract.

Equation 11.3 is one gradient descent step. It moves from an initial position, x , to a new position based on the value of the slope at the current position. Again η , the learning rate, governs how far we move.

Now that we have the equation, let's implement gradient descent. We'll plot Equation 11.2, pick a starting position, say $x = -0.9$, and iterate Equation 11.3, plotting the function value at each new position of x . If we do this, we should see a series of points on the function that move ever closer to the minimum position at $x = 1$. Let's write some code.

First, we implement Equation 11.2 and its derivative:

```
def f(x):  
    return 6*x**2 - 12*x + 3  
def d(x):  
    return 12*x - 12
```

Next, we plot the function, and then we iterate Equation 11.3, plotting the new pair, $(x, f(x))$, each time:

```
import numpy as np  
import matplotlib.pyplot as plt  
  
❶ x = np.linspace(-1,3,1000)  
    plt.plot(x,f(x))  
  
❷ x = -0.9  
    eta = 0.03  
❸ for i in range(15):  
    plt.plot(x, f(x), marker='o', color='r')  
    ❹ x = x - eta * d(x)
```

Let's walk through the code. After importing NumPy and Matplotlib, we plot Equation 11.2 ❶. Next, we set our initial x position ❷ and take 15 gradient descent steps ❸. We plot before stepping, so we see the initial x but do not plot the last step, which is fine in this case.

The final line ❹ is key. It implements Equation 11.3. We update the current x position by multiplying the derivative's value at x by $\eta = 0.03$ as the step size. The code above is in the file *gd_1d.py*. If we run it, we get Figure 11-1.

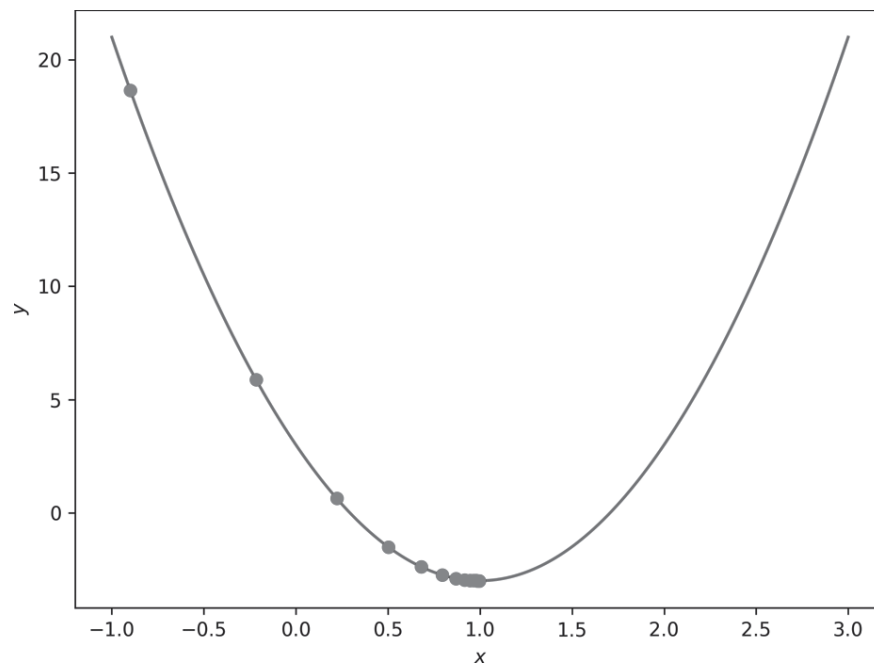


Figure 11-1: Gradient descent in one dimension with small steps ($\eta = 0.03$)

Our initial position, which we can think of as an initial guess at the location of the minimum, is $x = -0.9$. Clearly, this isn't the minimum. As we take gradient descent steps, we move successively closer to the minimum, as the sequence of circles moving toward it shows.

Notice two things here. First, we do get closer and closer to the minimum. After 14 steps, we are, for all intents and purposes, at the minimum: $x = 0.997648$. Second, each gradient descent step leads to smaller and smaller changes in x . The learning rate is constant at $\eta = 0.03$, so the source of the smaller updates to x must be smaller and smaller values of the derivative at each x position. This makes sense if we think about it. As we approach the minimum position, the derivative gets smaller and smaller, until it reaches zero at the minimum, so the update using the derivative gets successively smaller as well.

We selected the step size for Figure 11-1 to move smoothly toward the minimum of the parabola. What if we change the step size? Further along in *gd_1d.py*, the code repeats the steps above, starting at $x = 0.75$ and setting $\eta = 0.15$ to take steps that are five times larger than those plotted in Figure 11-1. The result is Figure 11-2.

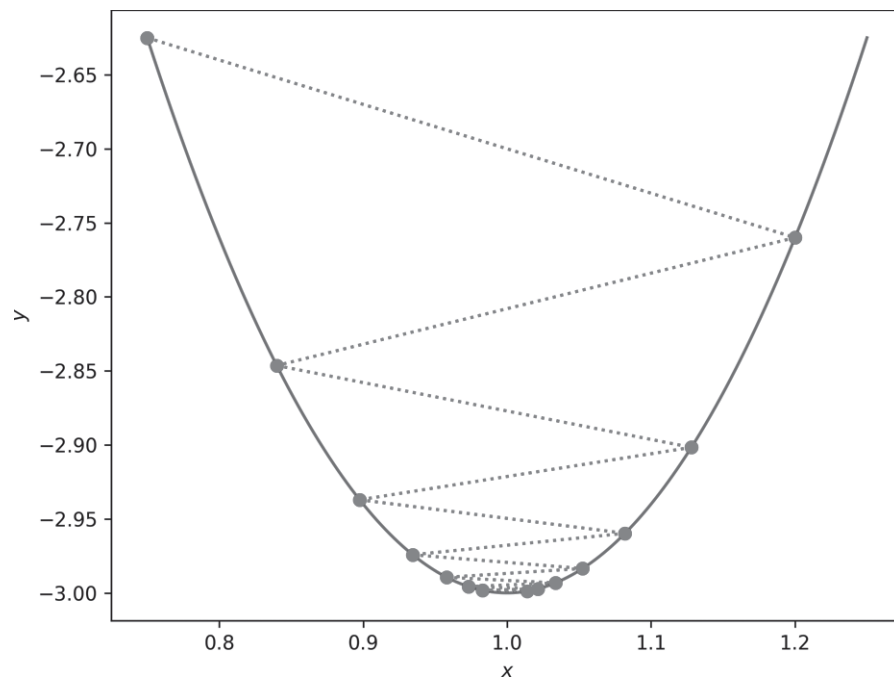


Figure 11-2: Gradient descent in one dimension with large steps ($\eta = 0.15$)

In this case, the steps overshoot the minimum. The new x positions oscillate, bouncing back and forth over the true minimum position. The dashed lines connect successive x positions. The overall search still approaches the minimum but takes longer to reach it, as the large step size makes each update to x tend to move past the minimum.

Small gradient descent steps move short distances along the function, whereas large steps move large distances. If the learning rate is too small, many gradient descent steps are necessary. If the learning rate is too large, the search overshoots and oscillates around the minimum position. The proper learning rate is not immediately obvious, so intuition and experience come into play when selecting it. Additionally, these examples fixed η . There's no reason why η has to be a constant. In many deep learning applications, the learning rate is not constant but evolves as training progresses, effectively making η a function of the number of gradient descent steps taken.

Gradient Descent in Two Dimensions

Gradient descent in one dimension is straightforward enough. Let's move to two dimensions to increase our intuition about the algorithm. The code referenced below is in the file *gd_2d.py*. We'll first consider the case where the function has a single minimum, then look at cases with multiple minima.

Gradient Descent with a Single Minimum

To work in two dimensions, we need a scalar function of a vector, $f(\mathbf{x}) = f(x, y)$, where, to make it easier to follow, we separate the vector into its components, $\mathbf{x} = (x, y)$.

The first function we'll work with is

$$f(x, y) = 6x^2 + 9y^2 - 12x - 14y + 3$$

To implement gradient descent, we need the partial derivatives as well:

$$\frac{\partial f}{\partial x} = 12x - 12$$

$$\frac{\partial f}{\partial y} = 18y - 14$$

Our update equations become

$$x \leftarrow x - \eta \frac{\partial f}{\partial x} = x - \eta (12x - 12)$$

$$y \leftarrow y - \eta \frac{\partial f}{\partial y} = y - \eta (18y - 14)$$

In code, we define the function and partial derivatives:

```
def f(x,y):
    return 6*x**2 + 9*y**2 - 12*x - 14*y + 3
def dx(x):
    return 12*x - 12
def dy(y):
    return 18*y - 14
```

Since the partial derivatives are independent of the other variable, we get away with passing only x or y . We'll see an example later in this section where that's not the case.

Gradient descent follows the same pattern as before: select an initial position, this time a vector, iterate for some number of steps, and plot the path. The function is 2D, so we first plot it using contours, as shown next.

```

N = 100
x,y = np.meshgrid(np.linspace(-1,3,N), np.linspace(-1,3,N))
z = f(x,y)
plt.contourf(x,y,z,10, cmap="Greys")
plt.contour(x,y,z,10, colors='k', linewidths=1)
plt.plot([0,0],[-1,3],color='k',linewidth=1)
plt.plot([-1,3],[0,0],color='k',linewidth=1)
plt.plot(1,0.777778,color='k',marker='+')

```

This code requires some explanation. To plot contours, we need a representation of the function over a grid of (x, y) pairs. To generate the grid, we use NumPy, specifically `np.meshgrid`. The arguments to `np.meshgrid` are the x and y points, here provided by `np.linspace`, which itself generates a vector from -1 to 3 of $N = 100$ evenly spaced values. The `np.meshgrid` function returns two 100×100 matrices. The first contains the x values over the given range, and the second contains the y values. All possible (x, y) pairs are represented in the return value to form a grid of points covering the region of $-1 \dots 3$ in both x and y . Passing these points to the function then returns z , a 100×100 matrix of the function value at each (x, y) pair.

We could plot the function in 3D, but that's difficult to see and unnecessary in this case. Instead, we'll use the function values in x , y , and z to generate contour plots. Contour plots show 3D information as a series of lines of equal z value. Think of lines around a hill on a topographic map, where each line is at the same altitude. As the hill gets higher, the lines enclose successively smaller regions.

Contour plots come in two varieties, as either lines of equal function value or shading over ranges of the function. We'll plot both varieties using a grayscale map. That's the net result of calling Matplotlib's `plt.contourf` and `plt.contour` functions. The remaining `plt.plot` calls show the axes and mark the function minimum with a plus sign. The contour plots are such that lighter shades imply lower function values.

We're now ready to plot the sequence of gradient descent steps. We'll plot each position in the sequence and connect them with a dashed line to make the path clear (see Listing 11-1). In code, that's

```

x = xold = -0.5
y = yold = 2.9
for i in range(12):
    plt.plot([xold,x],[yold,y], marker='o', linestyle='dotted', color='k')
    xold = x
    yold = y
    x = x - 0.02 * dx(x)
    y = y - 0.02 * dy(y)

```

Listing 11-1: Gradient descent in two dimensions

We begin at $(x, y) = (-0.5, 2.9)$ and take 12 gradient descent steps. To connect the last position to the new position using a dashed line, we track

both the current position in x and y and the previous position, $(x_{\text{old}}, y_{\text{old}})$. The gradient descent step updates both x and y using $\eta = 0.02$ and calling the respective partial derivative functions, dx and dy .

Figure 11-3 shows the gradient descent path that Listing 11-1 follows (circles) along with two other paths starting at $(1.5, -0.8)$ (squares) and $(2.7, 2.3)$ (triangles).

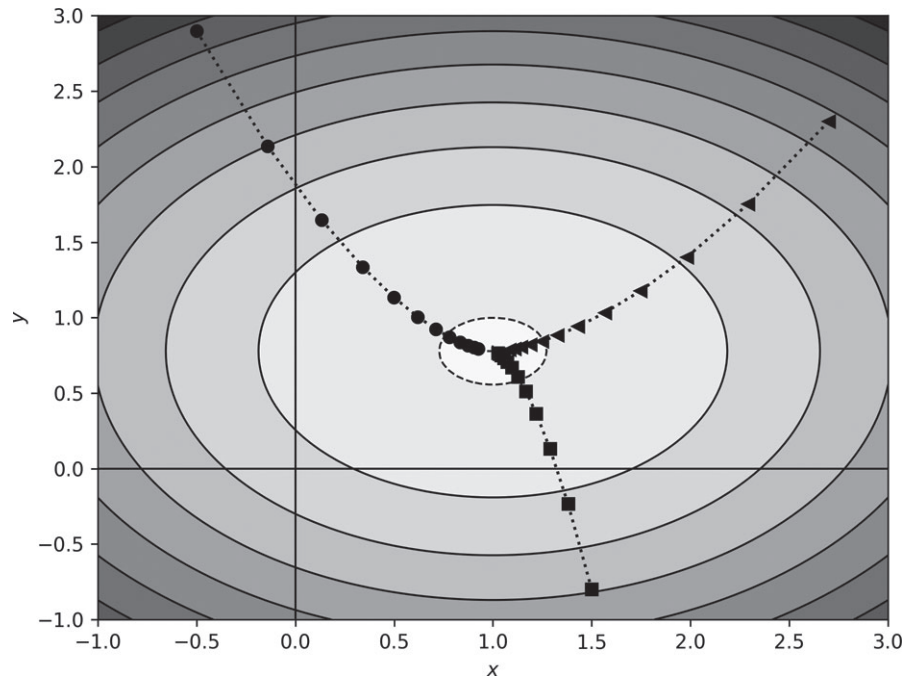


Figure 11-3: Gradient descent in two dimensions for small steps

All three gradient descent paths converge toward the minimum of the function. This isn't surprising, as the function has only one minimum. If the function has a single minimum, then gradient descent will eventually find it. If the step size is too small, many steps might be necessary, but they will ultimately converge on the minimum. If the step size is too large, gradient descent may oscillate around the minimum but continually step over it.

Let's change our function a bit to stretch it in the x direction relative to the y direction:

$$f(x, y) = 6x^2 + 40y^2 - 12x - 30y + 3$$

This function has partials $\partial f / \partial x = 12x - 12$ and $\partial f / \partial y = 80y - 30$.

Additionally, let's pick two starting locations, $(-0.5, 2.3)$ and $(2.3, 2.3)$, and generate a sequence of gradient descent steps with $\eta = 0.02$ and $\eta = 0.01$, respectively. Figure 11-4 shows the resulting paths.

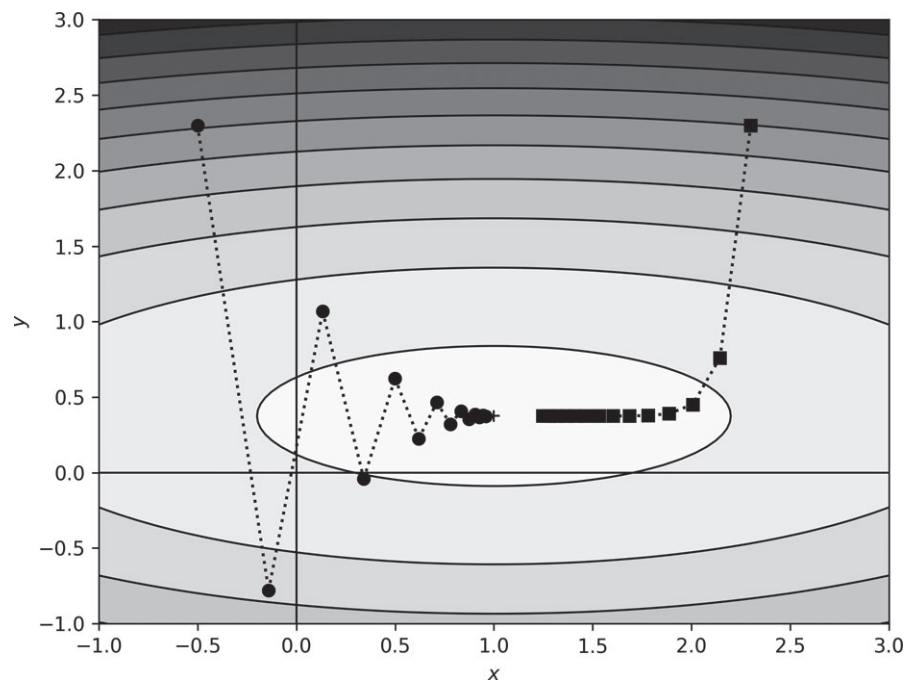


Figure 11-4: Gradient descent in 2D with larger steps and a slightly different function

Consider the $\eta = 0.02$ (circle) path first. The new function is like a canyon, narrow in y but long in x . The larger step size oscillates up and down in y as it moves toward the minimum in x . Bouncing off the canyon walls aside, we still find the minimum.

Now, take a look at the $\eta = 0.01$ (square) path. It quickly falls into the canyon and then moves slowly over the flat region along the canyon floor toward the minimum position. The component of the vector gradient (the x and y partial derivative values) along the x direction is small in the canyon, so motion along x is proportionately slow. There is no motion in the y direction—the canyon is steep, and the relatively small learning rate has already located the canyon floor, where the gradient is primarily along x .

What's the lesson here? Again, the step size matters. However, the shape of the function matters even more. The minimum of the function lies at the bottom of a long, narrow canyon. The gradient along the canyon is tiny; the canyon floor is flat in the x direction, so motion is slow because it depends on the gradient value. We frequently encounter this effect in deep learning: if the gradient is small, learning is slow. This is why the rectified linear unit has come to dominate deep learning; the gradient is a constant one for positive inputs. For a sigmoid or hyperbolic tangent, the gradient approaches zero when inputs are far from zero.